

RPC File Transfer System

Nguyen Mai Hung

ID: 23BI14180

December 4, 2025

1 Overview

This report outlines the implementation of a file transfer system based on Remote Procedure Calls (RPC). Using Python's `xmlrpc` library, the system enables reliable binary file transmission over HTTP. To handle large files efficiently, data is split into 64KB chunks and encoded in Base64 before transport.

2 System Workflow

The system operates on a synchronous client-server model. The client initiates a transfer session, uploads data sequentially, and finalizes the transaction.

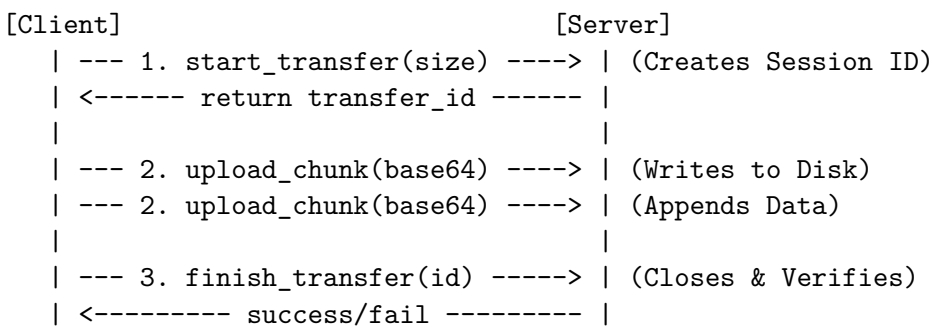


Figure 1: Transfer Logic Flow

3 Key Implementation

3.1 Server: Handling Chunks

The server writes chunks immediately to disk to minimize memory usage.

```
1 def upload_chunk(self, transfer_id, chunk_data, chunk_number):
2     transfer = self.active_transfers[transfer_id]
3     # Decode base64 back to binary and write to file
4     binary_data = base64.b64decode(chunk_data)
5     transfer['file_handle'].write(binary_data)
6     transfer['received'] += len(binary_data)
7     return {'status': 'success'}
```

3.2 Client: Sending Data

The client reads the source file in blocks, encodes them, and sends them via the proxy.

```
1 with open(filepath, 'rb') as f:
2     while True:
```

```
3     chunk = f.read(self.chunk_size) # Read 64KB
4     if not chunk: break
5
6     # Encode binary chunk to string for XML-RPC
7     encoded = base64.b64encode(chunk).decode('utf-8')
8     self.proxy.upload_chunk(transfer_id, encoded, chunk_num)
```

4 Usage

1. Start the Server

```
1 python3 server.py [host] [port]
```

2. Transfer a File

```
1 python3 rpc_file_transfer_client.py my_data.zip
```

5 Conclusion

The system successfully implements a robust file transfer mechanism. By using chunking and unique session IDs, it ensures large files are transferred correctly without overwhelming server memory.